

Language Learning OS

Offline-First Multilingual Learning Platform

Tamil ↔ English ↔ Hindi — Mobile, Desktop & Web

Engineering Specification

Version 1.0 — Draft for Development Kickoff

Prepared for: Rathin

Table of Contents

1. Overview & Vision	3
2. System Architecture	4
3. Repository / Folder Structure	7
4. Database Schema (Entity-Relationship Design)	9
5. Content JSON Schema	11
6. Three-Level Curriculum	13
7. Voice Pipeline	14
8. Offline Model & Packaging Strategy	15
9. UI Screen Map	17
10. Development Phases	19
11. Technology Stack (Locked)	20
12. Notes & Open Risks	21

1. Overview & Vision

This document is the frozen engineering specification for the Language Learning OS — a personal, offline-first language tutor covering Tamil, English, and Hindi in all directions. It is derived from an initial product research and architecture session, and is intended as the direct input for development (including handoff to Claude Code as a master build prompt).

1.1 What this product is

A single codebase, deployed to mobile (Android/iOS), desktop (Windows/macOS), and web (installable PWA), that teaches vocabulary, sentences, grammar, pronunciation, listening, speaking, and live conversation — entirely offline after a one-time download. There is no backend for core functionality; the internet is optional and used only for content/app updates.

1.2 What this product is not

- Not a simple dictionary or word list
- Not a translation app
- Not a chatbot wrapper around a cloud LLM
- Not "one identical binary everywhere" — shared core logic with platform-specific voice/AI adapters

1.3 Core product definition

Tamil → English → Hindi Learning OS, supporting all six translation directions (Tamil↔English, Tamil↔Hindi, English↔Hindi), plus pronunciation guidance explained through Tamil, word/sentence meaning, usage examples, grammar, conversation practice, listening, speaking, reading, writing, spaced-repetition revision, quizzes, offline voice, and progress tracking — unified in one application.

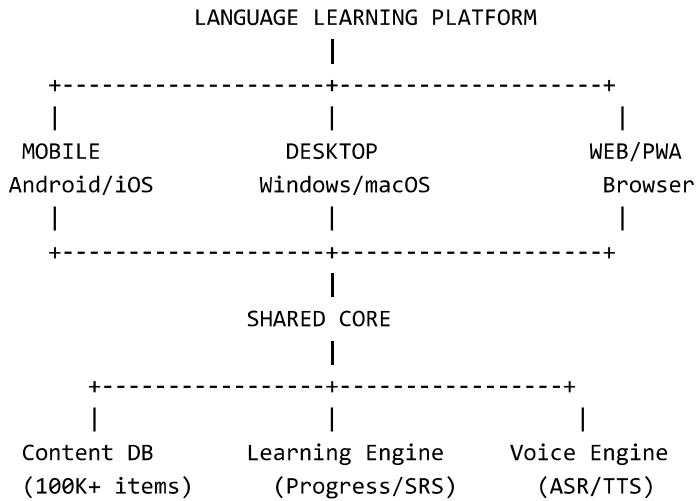
1.4 Guiding principle

NETWORK = OPTIONAL, NOT REQUIRED. If a feature depends on a cloud API (translation service, cloud AI, cloud sync) to function at all, it does not belong in the core product. AI is an optional accelerator layered on top of a fully functional non-AI core — not a dependency.

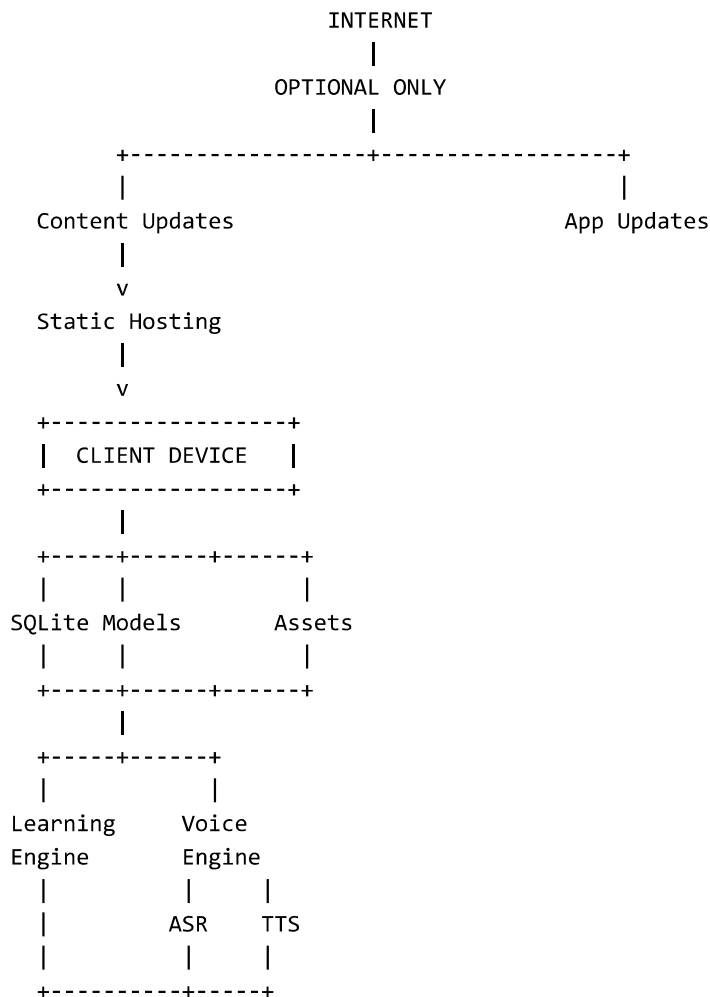
2. System Architecture

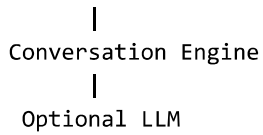
2.1 Platform layout

One shared core drives three platform targets:



2.2 Top-level data & control flow



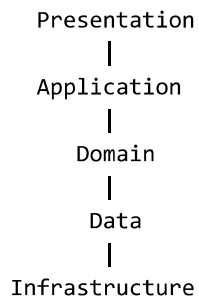


2.3 Why this architecture

Failure / constraint	Effect on product
Backend goes down	No effect — no backend in the core path
Internet unavailable	No effect — all core features work offline
User has no account	No effect — no login required
Content scales to 100K+ items	Handled by SQLite/FTS5 + lazy loading
New platform needed	Mobile, desktop, and web already supported by shared core
New language added	Additive via language packs; core engine unchanged

2.4 Layered / clean architecture

Each feature module follows a strict top-to-bottom dependency direction:



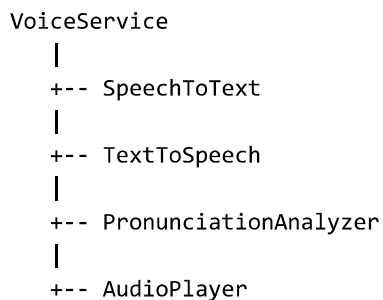
Example feature module layout:

```

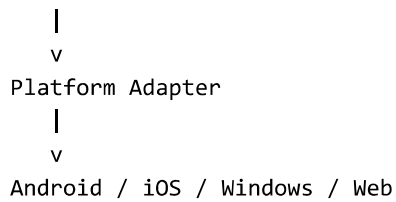
features/
  vocabulary/
    presentation/
    application/
    domain/
    data/
  
```

2.5 Voice architecture

Voice logic is isolated behind a service layer, never embedded directly in UI code, with a platform adapter beneath it:

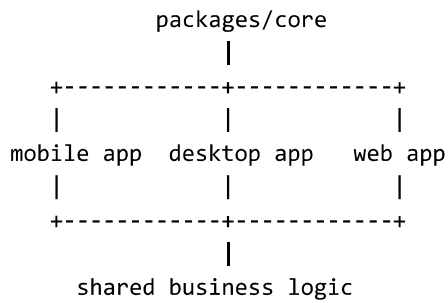


VoiceService



2.6 Monorepo principle

Shared business logic lives once, in `packages/core`, and is consumed by all three app targets — preventing logic duplication across mobile, desktop, and web.



3. Repository / Folder Structure

Git is the source of truth for source content and code. Compiled runtime databases are build artifacts, not committed source.

```
language-learning/
|
+-- apps/
|   +-- mobile/
|   +-- desktop/
|   +-- web/
|
+-- packages/
|   +-- core/
|   +-- content/
|   +-- learning/
|   +-- search/
|   +-- voice/
|   +-- conversation/
|   +-- ui/
|
+-- content/                (SOURCE — human-editable)
|   +-- english/
|   +-- hindi/
|   +-- tamil/
|   +-- conversations/
|   +-- grammar/
|   +-- quizzes/
|
+-- tools/
|   +-- content_compiler/
|   +-- validator/
|   +-- importer/
|   +-- database_builder/
|
+-- models/
|   +-- asr/
|   +-- tts/
|
+-- docs/
```

3.1 Feature-based UI structure

No single giant services/ folder. Each feature is self-contained:

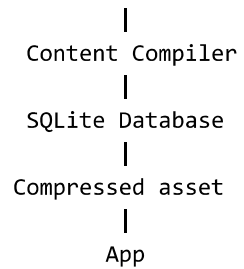
```
features/
+-- onboarding/
+-- home/
+-- vocabulary/
+-- sentences/
+-- grammar/
+-- lessons/
+-- practice/
+-- speaking/
+-- listening/
+-- conversation/
```

```
+-- quiz/
+-- revision/
+-- progress/
+-- search/
+-- settings/
```

3.2 Build-time content pipeline

Raw content is never shipped as JSON. It is compiled into SQLite at build time:

```
content/source/{words,sentences,grammar,conversations,quizzes}/ (JSON/CSV/YAML)
```



Git holds the source content. The compiled runtime artifacts (english.db, hindi.db, tamil.db, learning.db) are generated, not hand-edited:

```
Git → Content Compiler → SQLite → Release artifact → User device
```

4. Database Schema (Entity-Relationship Design)

Engine: SQLite with FTS5 full-text search. Two databases are kept logically separate: the Content Database (read-only, shipped with the app / updated via content packs) and the User Database (read/write, entirely local, holds progress).

4.1 Content database tables

Table	Key columns
languages	id, code, name, native_name
words	id, language_id, word, normalized_word, type, level, category, frequency, difficulty
translations	id, source_word_id, target_word_id, meaning, context
sentences	id, level, category, source_language, source_text
sentence_translations	sentence_id, language, translated_text
pronunciations	id, content_id, language, ipa, tamil_pronunciation, english_pronunciation, audio_asset
examples	id, word_id, sentence_id, context, meaning
grammar	id, level, topic, explanation, examples
conversations	id, level, scenario, title
conversation_lines	id, conversation_id, speaker, text, translation, expected_response
quiz_questions	id, level, type, question, answer
categories	id, name
levels	id, name, rank

4.2 User database tables

Table	Key columns
user_progress	content_id, ease_factor, interval, repetitions, last_review, next_review, mastery
user_settings	preferences, active languages, install packs, audio settings, etc.

4.3 Indexing & search strategy

- Indexes on: word, normalized_word, language, level, category, frequency
- FTS5 virtual tables across: English, Tamil, Hindi, Roman Hindi
- Cache layer for: recent searches, recent lessons, today's content
- Search must resolve native script, Tamil-approximate pronunciation, and transliterated input to the same result set (e.g. "sapidu" → சாபிடு / eat / खाना)

4.4 Relationship modeling beyond flat lookup

Words are connected to their inflections and common collocations so that learning one word surfaces related content automatically:

- Inflections: go → goes, going, went, gone
- Collocations: go home, go outside, go there, go to work, go shopping

4.5 Sense disambiguation (context engine)

A single English word can map to multiple, unrelated Tamil meanings (e.g. "bank" → வங்கி / கரை). Translations are therefore modeled per-sense, not per-word:

word -> sense -> context -> translation

5. Content JSON Schema

Every source content item carries full metadata so it can drive search, difficulty sequencing, and the multi-directional display simultaneously. Example item:

```
{
  "id": 10025,
  "level": "beginner",
  "category": "daily_life",
  "english": "Where are you going?",
  "tamil": "நீங்கள் எங்கே செல்லுகிறீர்கள்?",
  "hindi": "आप कहाँ जा रहे हैं?",
  "roman_hindi": "Aap kahan ja rahe hain?",
  "tamil_pronunciation": "ஆப் கடான் ஜா ரஹே ஹைன்",
  "difficulty": 1,
  "frequency": 0.92
}
```

5.1 What a single sentence expands into

A learner sees the same sentence rendered across six representations at once, e.g. "நான் வீட்டிற்கு போகிறேன்." ("I am going home."):

Representation	Value
English	I am going home.
English pronunciation (Tamil approx.)	ஐ ஆம் கோயிங் ஹோம்
Hindi	मैं घर जा रहा हूँ।
Roman Hindi	Main ghar ja raha hoon.
Hindi pronunciation (Tamil approx.)	மைன் கர் ஜா ரஹா ஹூன்
Audio	Bundled / on-device TTS

5.2 Word-level detail (example: "eat")

Field	Value
Tamil	சாப்பிடு / சாப்பிடுதல்
Hindi	खाना
Hindi pronunciation	கானா
English pronunciation	ஈட்
Part of speech	Verb
Difficulty	Beginner
IPA	/i:t/

5.3 Content pipeline (quality gate)

100K+ items is only useful if content quality holds. Every item passes through:

Raw Content -> Normalize -> Duplicate Detection -> Grammar Validation

-> Translation Validation -> Pronunciation Validation
-> Difficulty Classification -> Human Review -> Database

6. Three-Level Curriculum

Level 1 — Survival

Language needed to manage daily life.

- Greetings, Numbers, Family, Food, Travel, Shopping, Work, Time, Money
- Directions, Basic questions, Basic verbs, Basic adjectives

Target: 500–1,000 core items.

Level 2 — Everyday Fluency

- Daily conversation, Workplace, Phone calls, Social situations
- Questions, Past / Present / Future tense, Common grammar, Longer sentences

Target: 3,000–5,000 core items + thousands of sentences.

Level 3 — Natural Fluency

- Advanced conversation, Idioms, Phrasal verbs, Professional English
- Natural Hindi, Expressions, Storytelling, Opinions, Debates
- Work communication, Interviews, Presentations

Emphasis shifts from vocabulary to natural sentence formation.

6.1 Conversation scenarios (initial 20, expanding to 100+)

- 01 Greetings 02 Introduction 03 Family 04 Friends 05 Shopping
- 06 Restaurant 07 Travel 08 Airport 09 Hotel 10 Office
- 11 Interview 12 Phone call 13 Directions 14 Doctor 15 Bank
- 16 School 17 College 18 Workplace 19 Meeting 20 Emergency

6.2 Learning loop

Learn -> Practice -> Speak -> Quiz -> Mistake -> Review -> Mastery

6.3 Spaced repetition schedule

Day 1 -> Day 2 -> Day 4 -> Day 7 -> Day 14 -> Day 30 -> Day 60
(correct answer lengthens the interval; a miss resets/shortens it)

6.4 Adaptive sequencing example

If a learner gets "want" and "need" wrong on Day 1, tomorrow's review resurfaces those two words plus every sentence that uses them ("I want water.", "I need help.", "I want to go.", "I need to work."), rather than moving on blindly.

7. Voice Pipeline

The full loop:

```
MIC -> Speech Recognition -> Text -> Learning/Conversation Engine
    -> Response -> TTS -> Speaker
```

7.1 Speech-to-text (ASR)

- Engine: Whisper.cpp — supports Windows, macOS, iOS, Android, WebAssembly
- Web: runs via WASM; tiny/base models run roughly 2–3x real-time on modern browser CPUs — usable but not native-app-fast
- Android (API 31+): on-device SpeechRecognizer via createOnDeviceSpeechRecognizer() where the device supports it, with bundled Whisper as fallback
- iOS: on-device recognition via requiresOnDeviceRecognition = true (Apple notes on-device accuracy can be lower), with Whisper fallback

Platform adapter pattern: native on-device recognizer first, capability-detected; bundled Whisper.cpp engine as the universal fallback.

7.2 Text-to-speech (TTS)

- Piper — fast local neural TTS; includes Hindi voices (e.g. pratham, priyamvada) plus English and Malayalam
- Kokoro — 82M-parameter open-weight TTS model with Hindi support

Initial approach: benchmark English and Hindi against both Piper and Kokoro, select per-language.

7.3 Conversation engines

Engine A — Structured Conversation (100% reliable)

Scenario -> Intent -> Expected phrases -> Response

Covers the scenario library (shopping, restaurant, airport, office, interview, doctor, travel, etc.). Fast, deterministic, no model risk.

Engine B — Local AI Conversation (advanced mode)

Speech -> Whisper -> Local LLM -> Response -> TTS

Open-ended tutoring and correction (e.g. correcting "I am go market yesterday" to "I went to the market yesterday" with a grammar explanation, mirrored in Hindi and Tamil). Primarily a desktop-class experience; not forced onto mobile.

7.4 Pronunciation scoring

Word-level pronunciation feedback (e.g. per-word check marks plus an overall percentage) is a reasonable first-version feature. Phoneme-level scoring should not be marketed as highly accurate until backed by a proper pronunciation evaluation model.

8. Offline Model & Packaging Strategy

8.1 Offline capability matrix

Feature	Offline?
Vocabulary	Yes
Translation (from DB)	Yes
Search	Yes
Grammar	Yes
Lessons	Yes
Quiz	Yes
Progress	Yes
Spaced repetition	Yes
Pronunciation text	Yes
Audio	Yes
TTS	Yes
Speech recognition	Yes* (device/model dependent)
Conversation (structured)	Yes
Local AI conversation	Yes* (device/model dependent)
Cloud AI	No — out of scope
Cloud sync	No — out of scope
Content updates	Optional, not required

8.2 "100% offline" definition

A hard rule: if the app calls Google Translate, OpenAI, or any cloud API for a core function, it is not an offline app — regardless of what else runs locally. All models (ASR, TTS, optional LLM) must run on-device.

8.3 App size management

The realistic size pressure comes from bundled models (Whisper, TTS voices, optional LLM) and audio, not from the 100K-item content database itself. Mitigation: modular install packs selected at first run.

Pack	Approx. size
Core English content	80 MB
Hindi content pack	60 MB
English voice pack	40 MB
Hindi voice pack	30 MB
Speech (ASR) model	75 MB

Audio strategy: pre-generate and bundle compressed audio for the core 5–10K highest-frequency items; generate audio on-device via TTS for the long-tail of the 100K+ catalogue.

8.4 Content update mechanism (no backend required)

content-v1.db, content-v2.db, content-v3.db (versioned, static hosting)

App detects internet -> checks for new pack -> downloads

-> verifies checksum -> installs -> deletes old pack

No internet -> continues on currently installed content, unaffected

8.5 Local AI usage discipline

Local AI is applied selectively, not universally — routing every interaction through an LLM would be slow and battery-costly:

Interaction type	Routed to
Normal lookup	Database
Existing translation	Database
Lesson content	Database
Quiz	Database
Scripted conversation	Conversation engine (Engine A)
Open-ended explanation / correction	Local LLM (Engine B, optional)

8.6 Platform-tiered model strategy

Platform	Voice / AI footprint
Mobile	Content DB + learning engine + Whisper small/tiny + TTS + rule-based conversation + small local model (optional)
Desktop	Everything mobile has, plus a larger local LLM and higher-quality voice models
Web	Content DB + learning engine + PWA + WASM voice where practical

9. UI Screen Map

9.1 Home screen

Good Morning

Today's Goal: 15 min streak
 Continue Learning: [English Level 1]
 Today's Review: [24 words]
 Speaking Practice: [Start]
 Conversation: [Practice]
 Quick Search: [Search words...]

Design intent: the moment the app opens, "what do I do now?" must be answered immediately — no navigation hunting.

9.2 Word detail screen

```
+-----+
| EAT          [ 🎧 ] |
|              |
| சாப்பிடு     |
| खाना         |
|              |
| /i:t/   ஈட்  |
|              |
| Example      |
| I eat rice.   |
| நான் சாதம் சாப்பிடுகிறேன். |
| मैं चावल खाता हूँ। |
|              |
| [ 🎧 Listen ] [ 🗣️ Practice ] |
+-----+
```

9.3 Search screen

Search must resolve across scripts and transliteration to the same result set. Typing "சாப்பிட" or "kaana" both surface the eat / சாப்பிடு / खाना cluster, with related words, phrases, and a mini-lesson attached — search doubles as a learning entry point, not just a lookup.

9.4 Progress dashboard

Your Progress

English: 78%
 Hindi: 61%
 Words mastered: 1,284
 Sentences: 2,410
 Speaking: 76%
 Grammar: 68%
 Streak: 12 days

9.5 Gamification (secondary, not core)

- XP, streaks, levels, badges, daily goals, achievements — present, but never the primary loop

Core loop stays: Learn → Understand → Speak → Use. Not a Duolingo-style game-first clone.

9.6 Performance rules for UI/data

- App startup: splash → load only tiny metadata → home → lazy-load DB modules
- Never load all 100K records at startup — always indexed query returning a small page (e.g. 20 items)

10. Development Phases

Build incrementally — not everything at once. Data scale comes last, after the engine and UX are proven.

Phase	Scope
1. Foundation	Flutter setup, Clean Architecture, SQLite, content schema, theme, navigation, search
2. Core Learning	Vocabulary, sentences, translations, pronunciation text, levels, lessons, quiz, progress
3. Learning Engine	SRS, daily review, weak-word detection, adaptive lessons, statistics
4. Voice	TTS, STT, audio player, pronunciation practice
5. Conversation	Scenario engine, dialogue engine, speaking mode, role play
6. Local AI	GGUF models, llama.cpp integration, AI tutor, open conversation, correction, explanation
7. Web/Desktop Optimization	PWA, WASM voice, desktop packaging, model management
8. Massive Content Scale-Up	10K → 25K → 50K → 100K+ items

10.1 Minimum viable product (MVP)

The first shippable version needs only Tamil→English and Tamil→Hindi, with:

- 2,000 words
- 5,000 sentences
- 20 conversation scenarios
- 3 levels
- Pronunciation + audio
- Quiz + search
- Spaced repetition
- Fully offline, with progress tracking

This is a usable, complete product on its own. Content then scales 2K → 10K → 50K → 100K+ in later phases.

10.2 Future language expansion

Initial languages: Tamil, English, Hindi. The language-pack architecture (dictionary + pronunciation + grammar + TTS + learning content, all behind one interface) allows future languages — Malayalam, Telugu, Kannada, Arabic, French, Spanish — to be added without changing the core engine. Pronunciation is never hardcoded per-language; it lives in a generic `pronunciation_system` (script, romanization, IPA, native audio, learner pronunciation).

11. Technology Stack (Locked)

Area	Choice
Frontend framework	Flutter (mobile, desktop, web from one codebase)
Architecture	Clean Architecture, feature-first, MVVM / presentation-state, repository pattern, DI
State management	Riverpod
Database	SQLite + FTS5
Content pipeline	JSON/CSV/YAML source → Python compiler → SQLite
Offline model	100% local-first; network optional
Web delivery	Flutter Web + installable PWA, WASM where needed, IndexedDB/browser persistence
Speech-to-text (ASR)	Whisper.cpp (native + WASM), with OS on-device recognizers where available
Text-to-speech (TTS)	Piper / Kokoro
Local AI (optional)	llama.cpp + GGUF quantized models
Content tooling language	Python (build-time tooling only — not an application backend)
Native integration (as needed)	Kotlin (Android), Swift (iOS), C/C++ (low-level voice)
Backend	None required for core functionality
Hosting	Static hosting only, for optional app/content updates

11.1 Primary developer skill surface

Roughly 80–90% of the project is coverable with three languages: Dart (application + UI), SQL (database), and Python (content tooling/automation). Kotlin, Swift, and C/C++ are scoped narrowly to native platform integration and low-level voice, not general application logic.

12. Notes & Open Risks

- Content engineering and voice UX are the real bottlenecks — not database performance or raw data volume at 100K+ items.
- Phoneme-level pronunciation scoring needs a dedicated evaluation model before it can be marketed as accurate; ship word-level feedback first.
- Web voice (WASM Whisper) will be noticeably slower than native — set expectations accordingly and lean on native on-device recognizers on mobile.
- On-device speech recognition availability is device-dependent on both Android and iOS — always pair with the bundled Whisper fallback.
- Local LLM usage should stay desktop-weighted; do not force a large model onto mobile hardware.
- Every future language addition should be validated against the language-pack architecture before any core-engine changes are considered.

Next step: use this specification as the direct input for a master development prompt for implementation (e.g. via Claude Code), rather than starting implementation from an unstructured brief.